

Risposte Esercizi G2 — Esami 2024

Sistemi Operativi — Renzo Davoli & Alberto Montresor **Fonte:** presentazioni 00-intro.pdf – 08-sicurezza.pdf

Esame 17 gennaio 2024

a) Algoritmi preemptive o non-preemptive per sistemi interattivi?

I sistemi interattivi (desktop, web server, terminali) richiedono **basso response time**: l'utente si aspetta risposte rapide a eventi (click, input da tastiera, richieste di rete). Il criterio di valutazione chiave è il **tempo di risposta**, non il throughput.

Gli schedulers preemptive sono più adatti, per le seguenti ragioni:

1. **Limitazione della monopolizzazione della CPU**: un processo CPU-bound (calcolo intensivo) in uno scheduler non-preemptive blocca la CPU fino a quando non fa una system call o termina. L'utente vede il sistema "congelato" finché quel processo non rinuncia volontariamente alla CPU.
2. **Garanzia sul response time**: con Round Robin preemptive e quantum Q , ogni processo attende al massimo $(n-1) \times Q$ ms prima di ottenere la CPU (n = numero processi), indipendentemente dal comportamento degli altri. Con scheduler non-preemptive nessuna garanzia è possibile.
3. **Reattività agli eventi**: quando arriva un evento (pressione tasto, click mouse) che riattiva un processo interattivo, uno scheduler preemptive può immediatamente preemptare il processo corrente e servire l'evento. Con scheduler non-preemptive, l'evento viene processato solo alla prossima cessione volontaria della CPU.

Algoritmi preemptive adatti ai sistemi interattivi:

- **Round Robin**: fairness garantita, response time predicibile, semplice.
- **Priority scheduling preemptive con aging**: processi interattivi (alta priorità perché fanno I/O frequente) vengono serviti subito; aging previene la starvation dei processi batch.
- **Multi-Level Feedback Queue (MLFQ)**: i processi interattivi rimangono nelle code ad alta priorità (brevi burst CPU + I/O), i batch scendono a code inferiori.

Scheduler non-preemptive (FCFS, SJF cooperativo) sono adatti a sistemi batch dove il throughput è prioritario e non ci sono utenti che aspettano risposte in tempo reale.

b) SSD per la memoria virtuale: meno thrashing? Perché è sconsigliato?

L'uso di SSD riduce il rischio di thrashing?

Sì, parzialmente. Il thrashing si verifica quando la somma dei working set supera la RAM disponibile: il sistema passa più tempo a gestire page fault (caricare/salvare pagine su disco) che a

eseguire codice utile. La gravità del thrashing dipende dalla **latenza di accesso alla memoria secondaria**:

- Disco rotazionale: latenza casuale ~ 5–10 ms (seek + rotational delay).
- SSD: latenza casuale ~ 0.1 ms (100× più veloce, nessun seek meccanico).

Con SSD, ogni page fault viene risolto 100× più velocemente → il sistema può gestire un rate di page fault molto più alto prima che l'utilizzo CPU crolli a zero. La "soglia di thrashing" si sposta verso l'alto: il sistema sopravvive con un working set leggermente superiore alla RAM disponibile.

Tuttavia, la causa del thrashing è strutturale (domanda > RAM disponibile), non la velocità del disco. Anche con SSD ultrarapidi, se la domanda di memoria è molto superiore alla RAM, il thrashing si verifica ugualmente. La soluzione vera è aumentare la RAM o ridurre il grado di multiprogrammazione.

Perché gli SSD sono comunque sconsigliati per la memoria virtuale (swap):

1. **Cicli di scrittura limitati:** le celle flash NAND possono essere cancellate e riscritte un numero limitato di volte (da ~1000 per TLC NAND a ~100.000 per SLC). Lo swap è intensivo in scrittura: ogni page-out scrive 4KB. Su un sistema con thrashing attivo, uno SSD può degradarsi in pochi mesi.
2. **Write amplification:** per scrivere 4KB (dimensione di una pagina), il controller SSD deve spesso cancellare e riscrivere un intero blocco flash da 128–512KB (granularità di cancellazione). Il volume di scrittura fisico è multiplo di quello logico.
3. **Alternativa più efficiente:** con il costo attuale della RAM, è quasi sempre più conveniente aggiungere RAM (che elimina completamente il problema del paging) piuttosto che usare SSD per lo swap.
4. **Soluzioni migliori esistono:** zswap (compressione delle pagine in RAM prima di swapparle), zram (swap device in RAM compressa) riducono la necessità di swap su disco senza usurare lo SSD.

c) Almeno tre incongruenze rilevabili in ext3 con il controllo di coerenza

`fsck` (file system check) verifica la coerenza strutturale del file system confrontando le diverse strutture dati (bitmap, contatori, liste). Le principali incongruenze rilevabili in ext3 (e ext2) sono:

1. **Reference count errato nell'i-node (`i_links_count`):** il numero di hard link registrato nell'i-node non corrisponde al numero effettivo di directory entry che vi puntano. `fsck` conta le entry e confronta.
 - Troppo alto → space leak (i-node e blocchi non vengono mai liberati).
 - Troppo basso → deallocazione prematura dei blocchi (corruzione dati).
2. **Blocchi condivisi tra più i-node (cross-allocation / overlapping blocks):** lo stesso numero di blocco fisico appare nella lista blocchi di due i-node distinti. Scrittura su un file corrompe l'altro. `fsck` costruisce una mappa blocco → i-node e cerca duplicati.

3. Inconsistenza tra bitmap dei blocchi e liste blocchi degli i-node:

- Blocco marcato *libero* nella bitmap ma referenziato da un i-node → double-allocation futura.
 - Blocco marcato *occupato* nella bitmap ma non referenziato da alcun i-node → spazio perso irrecuperabilmente senza fsck.
4. **i-node orfani:** i-node con dati validi (non zero) e `i_links_count > 0` ma non referenziato da nessuna directory entry. Il file è inaccessibile. fsck li sposta in `/lost+found` con un nome numerico (il numero di i-node).
 5. **Dimensione del file inconsistente (i_size):** il campo `i_size` nell'i-node non corrisponde al numero effettivo di blocchi allocati nella lista blocchi.
 6. **Directory malformata:** entry di directory con lunghezze record (`d_rec_len`) errate, o entry che puntano a numeri di i-node fuori range o a i-node non allocati.
 7. **Superblock corrotto o inconsistente con il block group descriptor:** il numero totale di blocchi, di i-node, o la dimensione del blocco nel superblock non corrisponde alle strutture reali.

Nota su ext3 vs ext2: il journaling di ext3 garantisce la coerenza delle operazioni recenti in caso di crash, ma **non rileva** corruzione preesistente, errori hardware silenti, o bug del kernel che hanno scritto dati errati. fsck resta necessario per la verifica completa.

d) RAID60: struttura, dischi minimi, capacità, dischi coinvolti in scrittura

RAID60 = RAID0 di RAID6: K gruppi di dischi, ognuno organizzato internamente come RAID6; i dati sono distribuiti tramite striping RAID0 tra i gruppi.

Struttura con K gruppi da M dischi ciascuno (K×M dischi totali):

Parametro	Formula	Esempio (2 gruppi × 4 dischi)
Dischi totali	$K \times M$	8
Dischi minimi	$K=2 \text{ gruppi} \times M=4 \text{ dischi/gruppo}$	8 dischi (RAID6 richiede ≥ 4)
Dischi dati per gruppo	$M - 2$	2 per gruppo
Capacità totale	$K \times (M-2) \times C$	$2 \times 2 \times C = 4C$ (50% efficienza)
Fault tolerance	2 dischi per gruppo	Tolera 2 guasti in ogni gruppo

Minimo dischi: 8 (2 gruppi × 4 dischi). Con un solo gruppo sarebbe RAID6, non RAID60.

Capacità: con 8 dischi da C GB: $2 \times (4-2) \times C = 4C$ (4 dischi su 8 sono di parità, efficienza 50%).

Scrittura di un blocco dati — quanti dischi coinvolti?

Il blocco appartiene a uno specifico gruppo RAID6. In RAID6, aggiornare un blocco richiede:

1. Leggere il vecchio dato (disco del dato).
2. Leggere la vecchia parità P (disco parità P del gruppo).
3. Leggere la vecchia parità Q (disco parità Q del gruppo).
4. Scrivere il nuovo dato.
5. Scrivere la nuova parità P: $P_{\text{new}} = P_{\text{old}} \text{ XOR } d_{\text{old}} \text{ XOR } d_{\text{new}}$
6. Scrivere la nuova parità Q: aggiornata analogamente con i codici Reed-Solomon.

Totale: **6 operazioni I/O su 3 dischi** (dato + parità P + parità Q), tutti nello stesso gruppo RAID6. Lo striping RAID0 tra i gruppi non aggiunge overhead per un singolo blocco (va in un solo gruppo specifico).

Esame 13 febbraio 2024

a) Perché è necessario l'interval timer per lo scheduler Round Robin?

Round Robin richiede che ogni processo esegua per al massimo un **quantum di tempo** Q, dopo di che viene preemptato e il successivo processo nella ready queue prende la CPU.

Il problema senza timer hardware: Un processo in esecuzione in user mode non cede mai il controllo al kernel autonomamente (a meno che non faccia una system call o generi un'eccezione). Un processo CPU-bound che cicla su calcoli intensivi non ha mai bisogno del kernel → il kernel non riacquista mai il controllo → impossibile implementare la preemption.

Il ruolo dell'interval timer:

1. Prima di mandare in esecuzione un processo, il kernel **programma l'interval timer** per scattare dopo Q ms.
2. Il processo inizia l'esecuzione in user mode.
3. Allo scadere del quantum, il timer hardware genera un **interrupt**.
4. Il processore salta automaticamente all'interrupt handler del kernel (mode switch: user → kernel).
5. L'interrupt handler dello scheduler salva il contesto del processo corrente, lo rimette in fondo alla ready queue, e seleziona il prossimo processo.

Senza interval timer, l'unico modo per riottenere il controllo sarebbe attendere che il processo faccia una system call (I/O, `yield`, ecc.) — questo è esattamente lo scheduler

non-preemptive/cooperativo, non Round Robin. L'interval timer è il meccanismo hardware che rende possibile la preemption temporale.

b) Accesso diretto a file grandi: allocazione concatenata o indicizzata?

Allocazione concatenata (linked/chained): ogni blocco dati contiene il puntatore al blocco successivo (o la FAT è una tabella separata che implementa la catena).

Accesso al blocco N-esimo:

- Occorre partire dal primo blocco e seguire N-1 puntatori.
- Ogni puntatore richiede un accesso disco (o lettura della FAT).
- Complessità: **$O(N)$ accessi disco**.
- Per un file di 100MB con blocchi da 4KB: accedere al blocco 25000 richiede 24999 letture intermedie. **Inaccettabile per accesso diretto (seek).**

Allocazione indicizzata (i-node, indexed):

L'i-node contiene direttamente i puntatori ai blocchi, organizzati gerarchicamente:

- **Blocchi diretti** (es. 12 puntatori in ext2): accesso in 1 lookup di disco.
- **Singolo indiretto**: 1 accesso al blocco indice + 1 accesso al dato = 2 accessi.
- **Doppio indiretto**: 2 accessi indice + 1 dato = 3 accessi.
- **Triplo indiretto**: 3 accessi indice + 1 dato = 4 accessi.

Complessità massima: **$O(1)$** (al massimo 4 accessi disco indipendentemente dalla dimensione del file o dal numero del blocco cercato).

Risposta: l'allocazione **indicizzata** è enormemente più efficiente per l'accesso diretto a file grandi. L'allocazione concatenata pura richiede $O(N)$ accessi disco per il blocco N-esimo, rendendo lo $lseek$ su file grandi impraticabile.

(Nota: FAT mantiene la tabella in RAM, migliorando le prestazioni della concatenata, ma la scansione della catena rimane $O(N)$ in memoria — accettabile per file piccoli, penalizzante per file molto grandi.)

c) Algoritmo del Banchiere: cosa fa l'OS se la richiesta porta a stato unsafe?

Quando il processo P richiede le risorse R, l'algoritmo esegue:

1. **if** $R > \text{Need}[P]$ → errore (P sta richiedendo più del suo massimo dichiarato).
2. **if** $R > \text{Available}$ → P deve attendere (risorse fisicamente non disponibili).
3. **Simulazione tentativa:** $\text{Available} -= R, \text{Allocation}[P] += R, \text{Need}[P] -= R$.
4. **Algoritmo di sicurezza** sullo stato simulato: cerca una sequenza di completamento sicura per tutti i processi.

Se lo stato risultante è UNSAFE:

Il sistema operativo **annulla immediatamente la simulazione** (ripristina Available, Allocation, Need ai valori precedenti) e **sospende la richiesta**: il processo P viene bloccato in attesa, senza ricevere le risorse.

Perché? Uno stato unsafe non significa che il deadlock sia inevitabile, ma che *potrebbe* verificarsi:

non esiste garanzia che esista una sequenza di completamento sicura. L'algoritmo del banchiere è **conservativo**: preferisce non allocare (perdendo efficienza) piuttosto che rischiare un potenziale deadlock futuro.

P rimarrà sospeso finché le condizioni cambieranno (altri processi rilasciano risorse), e a quel punto la richiesta verrà rivalutata. Se la stessa richiesta diventa safe in seguito, verrà concessa.

d) RAID5 vs RAID50: capacità, fault tolerance, vantaggi

RAID50 = RAID0 di RAID5: K array RAID5 da M dischi ciascuno, con striping RAID0 tra i gruppi.

Con N dischi totali:

Parametro	RAID5 (N dischi)	RAID50 (K×M, K=2, M=N/2)
Capacità	$(N-1) \times C$	$K \times (M-1) \times C = (N-K) \times C$
Capacità (N=8)	7C	6C
Fault tolerance	1 disco qualsiasi	1 disco per gruppo (2 totali se in gruppi diversi)
Ricostruzione (1 disco guasto)	Legge tutti N-1 dischi rimanenti	Legge solo M-1 dischi del gruppo colpito
Prestazioni scrittura	Write penalty: 4 I/O	Migliore per striping tra gruppi

Capacità: RAID5 consente una capacità maggiore (un solo "disco" di parità equivalente per N dischi, vs K dischi di parità in RAID50). Con 8 dischi: RAID5 → 7C, RAID50 → 6C.

Fault tolerance: entrambi tollerano 1 disco guasto per gruppo, ma RAID50 con 2 gruppi può tollerare 2 guasti contemporanei **se in gruppi diversi**. Se 2 guasti sono nello stesso gruppo RAID5 → perdita dati. RAID5 standard tollera 1 guasto ovunque.

Vantaggi RAID50 su RAID5:

1. **Ricostruzione più rapida e sicura**: il rebuilding legge solo M-1 dischi (non tutti N-1). Meno tempo esposto al rischio di un secondo guasto durante la ricostruzione.
2. **Migliori prestazioni in lettura/scrittura**: striping tra più gruppi aumenta il parallelismo.
3. **Scalabilità**: si possono gestire array molto grandi con gruppi indipendenti.

Vantaggi RAID5 su RAID50:

1. **Più capacità**: con lo stesso numero di dischi, RAID5 utilizza più spazio.
 2. **Semplicità**: un solo array da gestire.
-

Esame 29 maggio 2024

a) Il DMA migliora le prestazioni della paginazione a richiesta?

Sì, il DMA (Direct Memory Access) migliora significativamente le prestazioni della memoria virtuale con paginazione a richiesta.

Senza DMA (PIO — Programmed I/O): Quando si verifica un page fault e si deve caricare una pagina da disco (tipicamente 4KB), la CPU deve:

- Interrogare il controller del disco per ogni parola di dati.
- Copiarla manualmente dal registro del controller in RAM.
- Ripetere per tutti i 4096 byte della pagina. La CPU è **occupata al 100%** per tutta la durata del trasferimento e non può fare nient'altro.

Con DMA:

1. La CPU avvia il trasferimento scrivendo nei registri DMA: indirizzo fisico sorgente (settore disco), indirizzo fisico destinazione (frame RAM), conteggio byte.
2. Il controller DMA trasferisce i dati **autonomamente** sul bus, senza coinvolgere la CPU.
3. Il processo che ha causato il page fault è sospeso (in attesa I/O).
4. La CPU è **libera** → lo scheduler esegue altri processi pronti.
5. Al termine del trasferimento, il DMA genera un interrupt: il kernel aggiorna la page table, sposta il processo in ready, riavvia l'istruzione che aveva causato il fault.

Beneficio per la memoria virtuale: Il DMA permette di **sovrapporre il trasferimento della pagina con l'esecuzione di altri processi**, aumentando l'utilizzo della CPU e il throughput del sistema. Senza DMA, ogni page fault "congela" la CPU per la durata del trasferimento disco — il che, con centinaia di page fault al secondo in un sistema sotto carico, porterebbe a un crollo dell'utilizzo CPU.

b) Un salt di un solo byte è sufficiente?

No, un salt di 1 byte non è sufficiente.

Scopo del salt: rendere impraticabili gli attacchi con dizionario e rainbow table costringendo l'attaccante a ricalcolare gli hash separatamente per ogni valore di salt.

Con salt di 1 byte (8 bit):

- Esistono solo $2^8 = 256$ possibili valori di salt.
- Un attaccante può precalcolare rainbow tables separate per **ognuno dei 256 salt possibili**.
- Il costo è 256 volte il costo delle rainbow table senza salt → ancora **computazionalmente fattibile** con hardware moderno (GPU che calcolano miliardi di hash al secondo).
- Una volta costruite le 256 tabelle (costo una-tantum), si cracca qualsiasi password del database istantaneamente.

Con salt adeguato (es. 16–32 byte = 128–256 bit):

- 2^{128} possibili valori di salt → precalcolare tutte le rainbow table è **computazionalmente impossibile**.
- L'attaccante deve eseguire brute force indipendentemente per ogni account.

Conclusione: 1 byte di salt riduce l'efficacia degli attacchi rainbow table di un fattore 256, ma non li rende impraticabili. Lo standard moderno richiede salt casuali di almeno 16 byte (128 bit), idealmente combinati con funzioni di hashing lente (bcrypt, Argon2, scrypt) che aumentano ulteriormente il costo del brute force.

c) Problemi di uno scheduler RR che dimezza il time slice ai processi CPU-bound

Il progettista vuole premiare i processi interattivi (che fanno I/O) penalizzando quelli CPU-bound (che consumano il quantum completamente senza I/O), dimezzando il loro prossimo time slice ad ogni quantum consumato completamente.

Problemi che emergono:

1. **Convergenza del time slice a zero (starvation operativa):** il time slice di un processo CPU-bound evolve come: $Q \rightarrow Q/2 \rightarrow Q/4 \rightarrow Q/8 \rightarrow \dots \rightarrow 0$. Dopo poche decine di cicli, il time slice scende sotto la soglia minima praticabile. Il processo ottiene frazioni di microsecondo di CPU per poi essere immediatamente preemptato → di fatto non progredisce mai.
2. **Overhead di context switch dominante:** il context switch ha un costo fisso di 1–10 μ s (salvataggio/ripristino registri, flush TLB, cache effects). Se il time slice scende sotto questo costo, il sistema spende più tempo a cambiare contesto che ad eseguire processi utili → il throughput effettivo crolla.
3. **Starvation dei processi CPU-bound:** con time slice bassissimi, i processi CPU-bound non completano mai lavoro significativo. Se ci sono costantemente processi interattivi in coda, i batch potrebbero non terminare mai.
4. **Classificazione errata di processi misti:** un processo che alterna fasi di calcolo intensivo con fasi I/O (es. compilatore, renderer) verrebbe penalizzato durante le fasi CPU anche se complessivamente non è un processo "greedy".
5. **Assenza di ripristino:** lo schema non prevede un meccanismo per aumentare il time slice quando un processo fa I/O. Un processo CPU-bound che occasionalmente fa I/O rimarrebbe con un time slice ridotto senza mai "recuperare".

Alternativa corretta: Multi-Level Feedback Queue (MLFQ) — i processi CPU-bound scendono a code a bassa priorità ma con **time slice più lunghi** (non più corti), evitando la starvation e il problema del context switch overhead.

d) Prevention vs Avoidance; l'allocazione gerarchica è Prevention o Avoidance?

Deadlock Prevention (Prevenzione): Approccio **statico**: si elimina strutturalmente almeno una delle 4 condizioni necessarie al deadlock (mutua esclusione, no preemption, hold-and-wait, attesa circolare). Il deadlock è impossibile per costruzione, indipendentemente dallo stato del sistema.

- Non richiede informazioni sulle richieste future.
- Spesso porta a utilizzo inefficiente delle risorse.
- Esempi: richiedere tutte le risorse in anticipo (elimina hold-and-wait), ordinare totalmente le risorse (elimina l'attesa circolare).

Deadlock Avoidance (Evitamento): Approccio **dinamico**: non elimina le condizioni, ma valuta ogni singola richiesta di risorse verificando se l'allocazione lascerebbe il sistema in uno stato **safe** (esiste una sequenza di completamento per tutti i processi). Richiede la conoscenza del credito massimo di ogni processo.

- Algoritmo del Banchiere come esempio.
- Overhead su ogni allocazione (esecuzione dell'algoritmo di sicurezza).
- Richiede informazioni a priori (massimo credito di ogni processo).

L'allocazione gerarchica (Hierarchical Resource Allocation) è Prevention.

Nell'allocazione gerarchica le risorse sono numerate totalmente ($R_1 < R_2 < \dots < R_n$). Ogni processo può richiedere risorse di classe i solo se non tiene già risorse di classe $\geq i$: le richieste devono procedere in **ordine strettamente crescente**.

Questo **elimina la condizione di attesa circolare**: se tutti i processi richiedono risorse in ordine crescente, nel grafo di allocazione non può formarsi un ciclo (un ciclo richiederebbe che esista $P_1 \rightarrow R_i \rightarrow P_2 \rightarrow R_j \rightarrow \dots \rightarrow P_n \rightarrow R_k \rightarrow P_1$ con $k < i$, impossibile con l'ordinamento). La restrizione è **strutturale e permanente**, non dipende dallo stato corrente del sistema → è Prevention.

Esame 25 giugno 2024

a) LeastUsed (LFU): quale hardware MMU è necessario?

LeastUsed / LFU (Least Frequently Used): la pagina vittima è quella con il minor numero totale di accessi dall'inizio della sua permanenza in memoria.

Hardware MMU necessario: Per implementare LFU, la MMU dovrebbe associare ad ogni frame (page table entry) un **contatore di accessi** che si incrementa automaticamente ad ogni riferimento alla pagina:

```
page_table_entry {
    frame_number: bits,
    valid: 1 bit,
    dirty: 1 bit,
    counter: N bits ← da incrementare ad ogni accesso
}
```

L'incremento deve avvenire in hardware ad ogni singolo accesso in memoria, perché:

- Gli accessi avvengono a frequenze nell'ordine di GHz.
- Un intervento software (trap al kernel) ad ogni accesso sarebbe impossibilmente costoso.

Possono esistere MMU con queste funzionalità?

In teoria sì, in pratica **no** per i sistemi general-purpose:

- Ogni accesso in memoria richiederebbe un'operazione di lettura-modifica-scrittura sul contatore nella page table → latenza aggiuntiva ad ogni accesso (cicli di memoria extra), inaccettabile.
- La page table è in memoria: aggiornare il contatore richiede un ulteriore accesso in memoria → potenzialmente ricorsivo.
- Il costo in transistor e latenza del bus è proibitivo.

Le MMU reali implementano solo il **reference bit** (1 bit: 0/1), molto più economico. Questo permette approssimazioni di LRU (Second Chance), non un vero LFU.

Problema aggiuntivo di LFU anche con hardware ideale: Una pagina molto acceduta durante la fase di avvio del processo (es. codice di inizializzazione) accumula un contatore alto e non viene mai rimpiazzata, anche se non viene più usata. L'aging (shift right periodico del contatore) mitiga questo problema ma aggiunge ulteriore complessità hardware.

b) Perché lo stack di valutazione non deve essere eseguibile?

Lo stack di valutazione (call stack) contiene: indirizzi di ritorno delle funzioni, variabili locali (inclusi buffer di dimensione fissa), parametri. È per definizione un'area **dati**, non codice.

Vulnerabilità senza protezione NX (No-eXecute):

Molte funzioni C copiano dati in buffer locali senza verificare i limiti (es. `strcpy`, `gets`). Un attaccante che controlla i dati in input può:

1. Riempire un buffer locale con **shellcode** (sequenza di istruzioni macchina che lancia una shell o esegue comandi arbitrari).
2. Sovrascrivere i byte oltre la fine del buffer, raggiungendo l'**indirizzo di ritorno** salvato nello stack frame.
3. Sovrascrivere l'indirizzo di ritorno con l'**indirizzo di inizio del buffer**.
4. Quando la funzione esegue `ret`, la CPU salta al buffer → **esegue il codice dell'attaccante** con i privilegi del processo vulnerabile (potenzialmente root).

Con stack non eseguibile (NX bit / DEP — Data Execution Prevention):

- La MMU marca le pagine dello stack come **non eseguibili** (bit NX nella PTE).
- Se il program counter punta a una pagina non eseguibile, la MMU genera una fault → il tentativo di eseguire shellcode dallo stack fallisce immediatamente.

- L'attaccante non può eseguire codice iniettato nello stack.

Principio: separazione dati/codice. Lo stack è dati: non dovrebbe mai essere necessario eseguire istruzioni da lì durante il normale funzionamento. La marcatura NX non costa nulla a runtime (solo 1 bit nella PTE) e fornisce una difesa robusta contro una classe intera di attacchi.

(Nota: tecniche avanzate come ROP — Return-Oriented Programming — aggirano NX riutilizzando frammenti di codice legittimo già in memoria; contro di esse si usa ASLR.)

c) Casi di utilizzo di RAID0

RAID0 (striping puro) non ha alcuna ridondanza: il guasto di un singolo disco dell'array distrugge irrimediabilmente tutti i dati. Nonostante questo limite critico, è utile nei seguenti scenari:

1. **Dati rigenerabili / temporanei:** directory /tmp, scratch space per rendering 3D, conversioni video, simulazioni scientifiche dove i dati sono calcolati e possono essere rigenerati dall'inizio. La perdita del disco porta alla rielaborazione, non alla perdita permanente di informazioni uniche.
 2. **Massima banda di I/O come requisito primario:** editing video 4K/8K in tempo reale, audio professionale multi-traccia ad alta risoluzione, applicazioni HPC dove la velocità di I/O è il collo di bottiglia e i dati originali risiedono altrove (NAS, cloud, tape).
 3. **Cache layer con ridondanza esterna:** come tier veloce in sistemi storage gerarchici dove i dati vengono periodicamente flushed su RAID protetto. La perdita del tier RAID0 è recuperabile dal tier sottostante.
 4. **Ambienti con backup continuo e RTO basso:** sistemi con backup real-time o near-real-time (replication, snapshot frequenti su storage separato) dove il Recovery Time Objective permette di tollerare la perdita dell'array sapendo che il backup è aggiornato.
 5. **Sviluppo e testing:** macchine di build/CI dove i dati (codice sorgente, artefatti di build) sono su repository remote (git). Le prestazioni di compilazione e test sono prioritarie.
 6. **Read-heavy workloads con copie multiple:** database di sola lettura replicati su più nodi — se un nodo RAID0 si guasta, gli altri nodi del cluster continuano a servire le query.
-

d) Spinlock in SMP (vedi esame 15/01/2025 domanda b)

Nei sistemi SMP, disabilitare gli interrupt è insufficiente (opera solo sulla CPU locale). Lo spinlock con test&set garantisce mutua esclusione atomica su tutto il bus memoria con overhead minimo per sezioni critiche brevissime del kernel.

Esame 22 luglio 2024

a) Il journaling risolve solo alcune incoerenze: quali non vengono gestite?

Il journaling (ext3, ext4, reiserfs) scrive un log delle operazioni **prima** di applicarle. In caso di crash, il replay del journal ripristina la coerenza strutturale delle operazioni interrotte.

Cosa il journaling gestisce: crash durante operazioni atomiche di modifica dei metadati (creazione/cancellazione file, aggiornamento directory, allocazione blocchi). La struttura del file system rimane coerente.

Cosa il journaling NON gestisce:

1. **Corruzione dei dati utente** (in modalità default `data=ordered` o `data=writeback`): il journaling di default protegge solo i **metadati** (strutture del file system), non i contenuti dei file. Se il sistema crasha dopo aver aggiornato i metadati ma prima di scrivere i dati effettivi, la struttura è coerente ma il file può contenere dati vecchi, nuovi o misti in modo imprevedibile. Solo la modalità `data=journal` protegge anche i dati, con significativa penalità di prestazioni.
2. **Silent data corruption (bit rot)**: corruzione dei dati nei settori del disco dovuta a degradazione fisica, bit flip cosmici o difetti del firmware del disco. Il disco non segnala errori ma restituisce dati sbagliati. Il journaling non verifica l'integrità dei dati con checksum → non rileva questo tipo di corruzione. (ZFS, btrfs usano checksum su ogni blocco proprio per questo.)
3. **Bug nel kernel / corruzione logica**: se un bug nel kernel scrive metadati errati (es. un reference count sbagliato, un puntatore a blocco errato), il journal registra fedelmente l'operazione errata. Il replay riproduce la corruzione: il file system è strutturalmente coerente ma logicamente sbagliato.
4. **Corruzione preesistente al crash**: il journal copre solo l'ultima sequenza di operazioni. Corruzione accumulata nel tempo (da sessioni precedenti, hardware difettoso) non viene né rilevata né corretta.
5. **Operazioni non atomiche a livello applicativo**: il journaling garantisce l'atomicità delle operazioni del file system, non delle operazioni applicative. Un'applicazione che aggiorna due file correlati (es. database senza WAL) può trovare uno dei due aggiornato e l'altro no dopo un crash.
6. **Guasto fisico del disco**: settori irrecuperabili, guasto del controller, failure del disco. Il journaling non può leggere/scrivere settori fisicamente danneggiati.

b) A cosa serve partizionare un disco?

Definizione: la partizione divide un disco fisico in sezioni logiche indipendenti, ognuna con proprio file system, struttura dati o sistema operativo. Il MBR o la GPT (GUID Partition Table) descrivono la mappa delle partizioni.

Motivazioni e casi pratici:

1. **Multi-boot:** installare più sistemi operativi sullo stesso disco fisico (es. Windows su C:, Linux su /dev/sda2). Ogni OS occupa e gestisce la propria partizione con il proprio file system. Il bootloader (GRUB, systemd-boot) sceglie quale avviare.
 2. **Separazione sistema/dati su Linux:** dividere / (root), /home (dati utente), /var (log, database), /tmp (temporanei) in partizioni distinte:
 - Se /var si riempie con log, il sistema (root) rimane operativo.
 - /home può essere preservata durante una reinstallazione del sistema.
 - Partizioni diverse possono avere opzioni di mount differenti (noexec per /tmp, nosuid per /home).
 3. **File system diversi per scopi diversi:** una partizione FAT32/exFAT per la condivisione dati con Windows (interoperabilità), una ext4 per Linux, una swap per la memoria virtuale, una EFI System Partition (FAT32) per il bootloader UEFI.
 4. **Isolamento di spazio e quote:** limitare lo spazio massimo usato da un servizio (es. database, log server) montando la sua directory su una partizione di dimensione fissa. Senza partizione separata, un servizio buggy potrebbe riempire l'intero disco compromettendo il sistema.
 5. **Recovery e ripristino:** una partizione di recovery separata (read-only, piccola) contiene strumenti diagnostici e l'immagine di ripristino. Inaccessibile agli utenti normali e protetta da scrittura accidentale.
 6. **Ottimizzazioni di allineamento e layout:** posizionare file system ad alta frequenza di accesso su partizioni agli estremi del disco (su HDD rotazionali le tracce esterne hanno maggiore densità lineare), o partizionare un SSD NVMe per separare carichi di lavoro con pattern di scrittura molto diversi.
-

c) Il tempo rimanente in SRTF può diventare negativo?

No, il tempo rimanente non può diventare negativo.

Spiegazione:

In SRTF (Shortest Remaining Time First), versione preemptive di SJF, il "tempo rimanente" è la **stima** del tempo di CPU ancora necessario al processo prima di completarsi.

Per i burst reali, la stima è aggiornata con la **media esponenziale (exponential averaging)**:

$$\tau_{n+1} = \alpha \times t_n + (1 - \alpha) \times \tau_n$$

dove:

- $t_n \geq 0$ è il burst di CPU effettivamente misurato (non può essere negativo).
- $\tau_n \geq 0$ è la stima precedente (non negativa per induzione).
- $\alpha \in [0, 1]$.

Poiché sia t_n che τ_n sono non negativi e i coefficienti sono non negativi con somma 1, $\tau_{n+1} \geq 0$ sempre. Per induzione (con $\tau_0 > 0$ come stima iniziale), tutte le stime successive sono non negative.

Intuitivamente: il tempo rimanente rappresenta quanto lavoro manca al processo per terminare. Questa quantità è per definizione ≥ 0 : o il processo ha ancora lavoro da fare (> 0) o è terminato ($= 0$, viene rimosso dalla coda). Non esiste un "tempo rimanente negativo" — significherebbe che il processo ha "terminato il lavoro in anticipo", concetto privo di senso.

Risposta: No, il tempo rimanente non può diventare negativo, né nella formulazione esatta (sistemi real-time con burst noti) né nella formulazione stimata (media esponenziale).

d) In quali casi è necessario compilare un kernel Linux specifico?

Linux è open source e ricompilabile. Compilare un kernel personalizzato è necessario o conveniente nei seguenti scenari:

1. **Hardware non supportato dal kernel della distribuzione:** driver proprietari o sperimentali per hardware industriale, FPGA, schede di acquisizione dati, hardware embedded personalizzato non ancora incluso nel mainline kernel.
 2. **Sistemi embedded con risorse limitate:** router, dispositivi IoT, sistemi di controllo industriale con poca RAM e flash storage. Un kernel minimale compilato per il solo hardware presente occupa ordini di grandezza meno spazio e RAM rispetto al kernel generico di una distribuzione.
 3. **Ottimizzazioni per architettura specifica:** il kernel delle distribuzioni è compilato per un'intera famiglia CPU (x86-64 generico). Ricompilando con `-march=native` si abilitano istruzioni specifiche (AVX-512, AES-NI) e ottimizzazioni di scheduling/prefetching per la CPU specifica.
 4. **Real-time Linux (PREEMPT_RT):** sistemi di controllo robotici, audio professionale (JACK), automazione industriale richiedono latenze deterministiche nell'ordine dei microsecond. La patch PREEMPT_RT trasforma Linux in un sistema hard real-time, non inclusa nel mainline kernel standard.
 5. **Patch di sicurezza urgenti:** applicare fix critici prima che siano distribuiti dalla distribuzione (window di esposizione ridotta), o applicare patch di hardening (grsecurity, KSPP) non incluse nel kernel stock.
 6. **Ricerca e sviluppo OS:** implementare e testare nuovi algoritmi di scheduling, nuovi gestori di memoria, nuovi filesystem o protocolli di rete sperimentali.
 7. **Abilitare funzionalità non compilate dalla distribuzione:** alcune funzionalità di sicurezza (kernel hardening, specifici LSM), debugging avanzato (kprobes, eBPF esteso), o supporto per filesystem sperimentali richiedono opzioni di configurazione non abilitate nei kernel delle distribuzioni per motivi di compatibilità o stabilità.
-

Esame 10 settembre 2024

a) RAID61 vs RAID16: livelli di fault tolerance

Convenzione di naming: RAID XY = RAID X di RAID Y.

- **RAID61 = RAID1 di RAID6**: si prende un array RAID6, poi lo si replica completamente con RAID1 (mirroring dell'intero array RAID6).
- **RAID16 = RAID6 di RAID1**: ogni "disco virtuale" visto da RAID6 è internamente un mirror RAID1.

RAID61 — configurazione minima: 2 gruppi × 4 dischi = 8 dischi fisici

Gruppo A (RAID6, 4 dischi): [D1][D2][P1][P2]
Gruppo B (RAID6, 4 dischi): [D1][D2][P1][P2] ← mirror completo di A

- RAID6 dentro ogni gruppo: tollera **2 guasti** all'interno del gruppo.
- RAID1 tra i gruppi: l'intero gruppo B è una copia di A → se l'intero gruppo A fallisce, B sopravvive.

Fault tolerance RAID61:

- Può perdere fino a **2 dischi qualsiasi** dal gruppo A (RAID6 li compensa) mentre il gruppo B è intatto.
- Può perdere l'**intero gruppo A** (tutti e 4 i dischi) e il sistema sopravvive grazie al mirror B.
- Worst case massimo: tutti i dischi del gruppo A (4) + 2 dischi del gruppo B = **6 dischi su 8** prima della perdita dei dati.

RAID16 — configurazione minima: 4 coppie RAID1 = 8 dischi fisici

RAID6 virtuale: [V1][V2][V3][V4]
V1 = RAID1(disco1a, disco1b)
V2 = RAID1(disco2a, disco2b)
V3 = RAID1(disco3a, disco3b) ← parità P
V4 = RAID1(disco4a, disco4b) ← parità Q

- RAID1 per ogni coppia: tollera **1 guasto fisico** per coppia (perde un disco, la coppia sopravvive).
- RAID6 sopra: tollera il fallimento completo di **2 coppie** (= 2 dischi virtuali).

Fault tolerance RAID16:

- Può perdere **1 disco fisico** da ciascuna delle 4 coppie (RAID1 compensa) = 4 guasti distribuiti.
- Può perdere **entrambi i dischi** di qualsiasi 2 coppie (RAID6 compensa 2 virtual disk failure) = 4 guasti concentrati.
- Combinazione: 2 coppie completamente guaste + 1 disco dalle coppie rimanenti = fino a **6 dischi su 8**.

Confronto:

	RAID61	RAID16
Dischi minimi	8	8
Tolleranza massima	Fino a 6/8 dischi (pattern dipendente)	Fino a 6/8 dischi (pattern dipendente)
Protezione singolo disco	RAID1 + RAID6	RAID1 per coppia
Protezione perdita di gruppo	Sì (mirror intero)	No (RAID6 su coppie)

b) FAT su disco rotazionale per /tmp: buona o cattiva idea?

Cattiva idea, per molteplici ragioni:

- 1. FAT è altamente soggetto a frammentazione:** /tmp è caratterizzato da creazione e cancellazione frequente di file di dimensioni variabili. FAT usa allocazione concatenata con blocchi sparsi: dopo molte operazioni, i file risultano frammentati su cilindri lontani del disco, aumentando il seek time (critico sui dischi rotazionali).
- 2. FAT non ha journaling:** Un crash di sistema con operazioni su /tmp in corso lascia il file system in uno stato inconsistente. Il riavvio richiede una scansione completa (scandisk/chkdsk) → tempi di boot allungati. Su Linux, /tmp viene acceduto da molti servizi di sistema all'avvio.
- 3. Ricerca di blocchi liberi inefficiente:** FAT usa una lista concatenata per registrare blocchi liberi e catene dei file. Trovare blocchi liberi richiede la scansione di tutta la tabella FAT — $O(n)$ nel numero di blocchi della partizione. Per /tmp con molte operazioni, questo diventa un collo di bottiglia.
- 4. Assenza di permessi UNIX:** FAT non supporta UID, GID, bit di permesso (rwx) né lo **sticky bit**. /tmp **deve** avere lo sticky bit abilitato (`chmod +t /tmp`): garantisce che solo il proprietario di un file possa cancellarlo, anche se tutti hanno permesso di scrittura sulla directory. Senza sticky bit, qualsiasi utente potrebbe cancellare i file temporanei degli altri processi.
- 5. Prestazioni intrinseche del disco rotazionale:** /tmp ha pattern di accesso casuale (file piccoli, molte operazioni). I dischi rotazionali sono lenti per I/O casuali (5–10 ms per accesso). La combinazione FAT + disco rotazionale + accesso casuale è la peggiore possibile per /tmp.

Soluzioni corrette:

- **tmpfs** (standard moderno): file system in memoria RAM, velocissimo, non persiste tra i riavvii (comportamento corretto per /tmp).
 - Partizione **ext4** (con journaling) su SSD se la persistenza tra riavvii è necessaria.
-

c) Interrupt nidificati: benefici e difficoltà

Gli **interrupt nidificati** permettono che un interrupt handler possa essere interrotto da un interrupt di priorità più alta, creando una pila di contesti di gestione.

Benefici:

1. **Reattività agli interrupt critici:** un interrupt ad alta priorità (timer real-time, segnale di emergenza hardware) viene gestito immediatamente, anche se si sta già gestendo un interrupt a bassa priorità (es. I/O disco). Senza nidificazione, il secondo interrupt attende il completamento del primo → latenza potenzialmente arbitraria.
2. **Throughput di gestione interrupt migliorato:** handler di interrupt diversi (rete, disco, tastiera, timer) possono sovrapporsi temporalmente. Con handler non interrompibili, un handler lento serializza tutti gli interrupt → degrado complessivo.
3. **Supporto per sistemi real-time:** fondamentale nei sistemi hard real-time dove il Worst-Case Response Time (WCRT) di interrupt critici deve essere bounded e minimo. Con nidificazione, il WCRT dipende solo dagli interrupt di priorità superiore, non da quelli di pari o inferiore priorità.

Difficoltà implementative:

1. **Gestione dello stack:** ogni interrupt nidificato consuma stack per salvare il contesto (registri, PC, PSW). Con nidificazione profonda, lo stack kernel può andare in overflow. Soluzioni: stack separati per interrupt handler (ogni livello di priorità ha il proprio stack), o dimensionamento conservativo dello stack.
2. **Reentrancy del codice kernel:** il codice dell'interrupt handler deve essere completamente rientrante. Non può usare variabili globali non protette: un handler può essere interrotto a metà da un handler dello stesso tipo (es. due interrupt di rete).
3. **Sezioni critiche all'interno degli handler:** se l'handler A modifica una struttura dati condivisa e viene interrotto dall'handler B che modifica la stessa struttura, si hanno race condition. Occorre usare spinlock o mascherare selettivamente gli interrupt durante le sezioni critiche — aumentando la latenza degli interrupt mascherati.
4. **Gestione delle priorità:** serve una gerarchia di priorità ben definita e un controller degli interrupt (es. APIC su x86) che mascheri automaticamente gli interrupt di priorità $\leq$ a quella corrente durante la gestione.
5. **Complessità di debug:** il flusso di esecuzione con interrupt nidificati è difficilissimo da tracciare. Race condition e bug di timing sono non deterministici e difficili da riprodurre in ambiente di test.

d) Aumentare la memoria secondaria della VM aumenta il rischio di thrashing?

Sì, paradossalmente aumentare lo spazio swap può **aumentare** il rischio di thrashing.

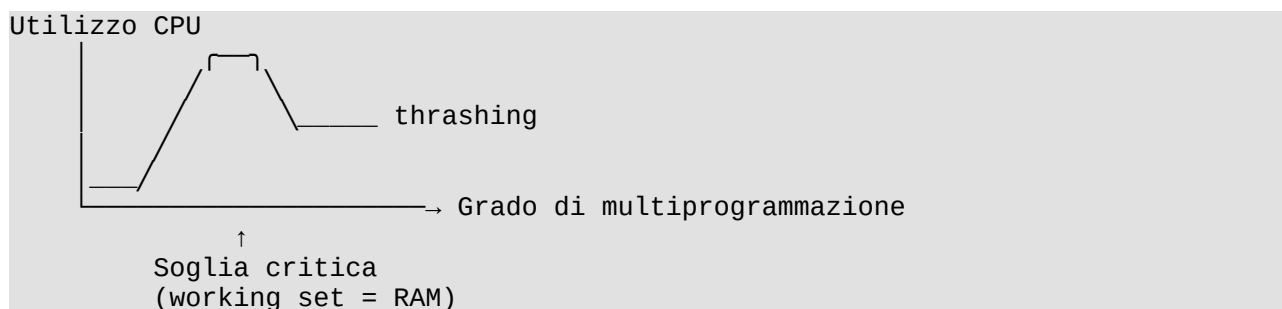
Meccanismo del thrashing: Il thrashing si verifica quando la somma dei working set di tutti i processi attivi supera la RAM disponibile. Il sistema entra in un ciclo patologico: ogni processo, per

eseguire, causa page fault e sottrae frame agli altri processi, i quali a loro volta causano page fault → la CPU passa quasi tutto il tempo a gestire page fault anziché a eseguire codice utile. L'utilizzo CPU crolla nonostante tutti i processi siano "pronti".

Come lo swap influenza il grado di multiprogrammazione:

1. Con **poco swap**: quando la RAM si esaurisce, i processi non possono essere mantenuti parzialmente in memoria. Il sistema termina processi (OOM killer) o ne impedisce la creazione. Il grado di multiprogrammazione è limitato naturalmente.
2. Con **molto swap**: i processi possono avere molte pagine su disco. Il sistema può mantenere **molti più processi contemporaneamente attivi** in RAM (anche se con poche pagine ciascuno). Il grado di multiprogrammazione aumenta.
3. **Superamento della soglia critica**: se il grado di multiprogrammazione supera il punto in cui la somma dei working set eccede la RAM, inizia il thrashing. Con più swap, si può arrivare ben oltre questa soglia senza che il sistema termini i processi.

Illustrazione con il grafico CPU utilization vs multiprogramming degree:



Più swap sposta la soglia verso destra (più processi possibili), ma non aumenta la RAM disponibile. Se il sistema non controlla il grado di multiprogrammazione con il modello del Working Set, si rischia di operare nella zona di thrashing.

Soluzione: il sistema operativo deve usare il **modello del Working Set** (o la frequenza di page fault) per controllare il grado di multiprogrammazione: ammettere nuovi processi solo se $\sum W(\Delta, P_i) \leq \text{frame disponibili}$.

Risposte Esercizi G2 — Esami 2025

Sistemi Operativi — Renzo Davoli & Alberto Montresor **Fonte:** presentazioni 00-intro.pdf – 08-sicurezza.pdf

Esame 15 gennaio 2025

a) Numero di link errato in un i-node ext2: conseguenze

Ogni i-node ext2 contiene il campo `i_links_count`, che conta quanti directory entry (hard link) puntano a quell'i-node. Quando scende a zero il kernel libera l'i-node e i blocchi dati.

Caso 1 — Contatore troppo alto (maggiore del reale): Quando il processo rimuove l'ultimo hard

link effettivo con un link, il contatore non scende a zero (valore errato). Il kernel non dealloca mai l'i-node né i blocchi dati. Il file non è più raggiungibile da nessun percorso, ma occupa spazio disco permanentemente (**space leak**). Il sistema rimane utilizzabile ma con spazio sprecato. fsck rileva l'incoerenza confrontando il contatore nell'i-node con il numero reale di directory entry che vi puntano, e può correggere il valore.

Caso 2 — Contatore troppo basso (minore del reale): Alla prima operazione un link che porta il contatore (errato) a zero, il kernel **dealloca l'i-node e i blocchi dati** anche se esistono ancora altri hard link validi che puntano al file. I processi che continuano ad accedere tramite i link superstiti leggono o scrivono blocchi già liberati, che possono essere stati riassegnati ad altri file. Il risultato è **corruzione dei dati** di entrambi i file coinvolti, potenzialmente irrecuperabile.

b) Perché usare spinlock (test&set) nei kernel SMP?

Nei sistemi monoprocesso la mutua esclusione si ottiene banalmente **disabilitando gli interrupt** sulla singola CPU: nessun altro thread può eseguire finché gli interrupt sono spenti.

Nei sistemi **multiprocesso simmetrici (SMP)** questa tecnica è insufficiente per due ragioni fondamentali:

1. **Disabilitare gli interrupt è locale alla CPU:** ogni processore ha il proprio registro di controllo degli interrupt. Disabilitarli sulla CPU A non impedisce alla CPU B di eseguire contemporaneamente codice kernel e accedere alle stesse strutture dati condivise in memoria. La mutua esclusione **non è garantita**.
2. **Le sezioni critiche del kernel sono brevissime:** usare semafori (che sospendono il processo) comporta un context switch completo. Per sezioni critiche di poche decine di istruzioni, il costo del context switch è enormemente superiore al lavoro da proteggere.

Lo spinlock risolve entrambi i problemi:

- test&set (o compare&swap) è un'**operazione atomica garantita dall'hardware** sull'intero bus di memoria: nessuna altra CPU può interferire mentre l'operazione è in corso.
- La CPU che trova il lock occupato esegue **busy-wait** (spin): cicla finché il lock si libera, senza context switch.
- Per sezioni critiche molto brevi (poche istruzioni), il tempo di spin è trascurabile.

Vincolo: gli spinlock sono appropriati *solo* per sezioni critiche brevissime. Per attese potenzialmente lunghe si usano mutex con sleep/wake, perché il busy-wait prolungato spreca interi cicli CPU.

c) Perché il second chance (orologio) è preferito a LRU nei sistemi reali?

LRU (Least Recently Used) è un algoritmo a stack (il set di pagine con m frame è sempre sottoinsieme di quello con $m+1$ frame), quindi non soffre dell'anomalia di Belady e approssima ottimalmente il comportamento futuro basandosi sul passato recente.

Il problema di LRU è l'implementazione hardware:

- Per sapere quale pagina è stata usata meno di recente occorre o un **timestamp per ogni accesso** (ogni riferimento in memoria aggiorna un registro hardware) oppure una **lista doppiamente concatenata** spostata ad ogni accesso.
- Entrambe richiedono un intervento hardware ad ogni singolo riferimento in memoria, che avviene miliardi di volte al secondo. Nessuna MMU commerciale implementa questo meccanismo perché:
 - Ogni accesso in memoria richiederebbe un ulteriore accesso per aggiornare la struttura LRU (ciclo infinito).
 - Il costo in transistor e latenza sarebbe proibitivo.

Second chance (algoritmo dell'orologio):

- Usa un solo **reference bit** per frame, che la MMU imposta a 1 ad ogni accesso — questa funzionalità è standard su tutte le MMU moderne.
- L'algoritmo scorre i frame in ordine circolare: se reference bit = 1 → lo azzera e avanza; se reference bit = 0 → seleziona la pagina come vittima.
- Complessità ammortizzata $O(1)$, nessun accesso aggiuntivo in memoria per ogni riferimento.
- **Non è un algoritmo a stack** (può in teoria soffrire di anomalia di Belady), ma in pratica approssima molto bene LRU.

Conclusione: second chance è preferito a LRU nei sistemi reali perché è **implementabile con hardware standard** (solo il reference bit) a costo trascurabile, mentre il vero LRU richiederebbe hardware dedicato troppo costoso. Il vantaggio teorico di LRU (stack property) non giustifica il costo implementativo.

d) Revoca di capability vs revoca in ACL

ACL (Access Control List): il diritto di accesso è memorizzato centralmente nella lista associata alla risorsa. Revocare il diritto significa rimuovere una entry dalla lista → operazione $O(1)$, immediata, in un unico punto.

Capability: un token (struttura dati o oggetto crittografato) che conferisce al possessore il diritto di accedere a una risorsa. Il diritto è **incorporato nel token stesso**, che può essere copiato e distribuito liberamente.

Perché la revoca è difficile:

	ACL	Capability
Dove risiede il diritto	Centralizzato nella risorsa	Distribuito presso ogni possessore
Revoca	Rimuovere una entry → $O(1)$	Trovare e invalidare tutte le copie → difficile

- La capability può essere stata **copiata**: un processo la riceve tramite `fork`, message passing, ecc. Non esiste un registro centrale di chi la possiede.
- **Non c'è un punto unico di controllo**: per revocarla occorre propagare la revoca a tutti i possessori, ma il sistema non sa quanti e dove siano.

Soluzioni parziali (tutte con overhead aggiuntivo):

1. **Capability indirette**: la capability punta a un oggetto intermedio; per revocare si invalida quell'oggetto (aggiunge indirectione ad ogni accesso).
2. **Timestamp/versione**: la capability include un numero di versione; il kernel incrementa la versione alla revoca (richiede confronto ad ogni accesso).
3. **Lista globale kernel**: il kernel registra tutte le capability emesse (overhead di gestione, punto di contesa).

In tutti i casi la revoca è più costosa e complessa rispetto a una ACL.

Esame 11 febbraio 2025

a) Aging: come funziona per evitare la starvation?

Starvation si verifica in uno scheduler a priorità quando un processo a bassa priorità non viene mai schedulato perché continuano ad arrivare processi a priorità più alta.

Aging è la tecnica che risolve questo problema **aumentando progressivamente la priorità** di un processo in proporzione al tempo che trascorre in attesa nella ready queue senza ottenere la CPU:

$$\text{priorità_effettiva}(t) = \text{priorità_base} + f(\text{tempo_di_attesa})$$

dove f è una funzione crescente (tipicamente lineare). Dopo un tempo sufficientemente lungo, anche un processo con priorità base bassissima raggiunge una priorità abbastanza alta da essere selezionato dallo scheduler.

Proprietà garantita: con aging ogni processo ottiene la CPU entro un tempo finito (upper bound dipende da priorità_base e velocità dell'aging), eliminando completamente la starvation. Quando il processo ottiene la CPU e viene eseguito, la sua priorità viene reimpostata al valore base.

b) Può esistere un controller DMA che non genera interrupt?

Sì, può esistere, ma risulta molto inefficiente.

Un controller DMA senza interrupt significherebbe che al termine del trasferimento dati non viene inviato alcun segnale alla CPU. Il device driver non può sapere quando il trasferimento è completato tramite notifica asincrona.

Come deve essere costruito il relativo device driver:

Il driver deve ricorrere al **polling**: dopo aver avviato il trasferimento DMA (scrivendo gli indirizzi sorgente/destinazione e il conteggio nei registri DMA), deve periodicamente leggere un **registro di**

stato del controller per verificare se il trasferimento è terminato. Le opzioni sono:

1. **Busy-wait puro:** il driver entra in un ciclo che legge continuamente il registro di stato finché il bit di completamento non è impostato. Spreca completamente la CPU durante il trasferimento, annullando il vantaggio del DMA rispetto al PIO (Programmed I/O).
2. **Polling periodico via timer:** il driver avvia il DMA, poi schedula un timer che si riattiva ogni X ms per controllare il registro di stato. Meno spreco ma latenza di rilevamento variabile.

Conclusione: un DMA senza interrupt è teoricamente possibile ma contraddice lo scopo del DMA stesso, che è liberare la CPU durante il trasferimento. Il binomio DMA + interrupt è la soluzione corretta per permettere alla CPU di fare altro mentre il trasferimento avviene.

c) Quanti dischi sono coinvolti da una scrittura di un blocco in RAID5?

In RAID5 con N dischi i dati sono distribuiti a strip su tutti i dischi, con la parità distribuita in modo che ogni disco funge a rotazione da disco di parità per una stripe.

La scrittura di un singolo blocco richiede 4 operazioni su 2 dischi:

Operazione	Disco coinvolto	Motivo
1. Lettura vecchio dato	Disco dati (disco <i>i</i>)	Serve per calcolare la nuova parità
2. Lettura vecchia parità	Disco parità (disco <i>p</i>)	Serve per calcolare la nuova parità
3. Scrittura nuovo dato	Disco dati (disco <i>i</i>)	Aggiornamento effettivo
4. Scrittura nuova parità	Disco parità (disco <i>p</i>)	$\text{nuova_parità} = \text{vecchia_parità} \text{ XOR } \text{vecchio_dato} \text{ XOR } \text{nuovo_dato}$

Sono coinvolti 2 dischi (il disco del dato e il disco della parità corrispondente alla stripe), con **4 operazioni I/O totali** (2 letture + 2 scritture). Questo è noto come **write penalty** di RAID5 e spiega perché le scritture in RAID5 sono significativamente più lente rispetto a RAID0.

d) Quale problema risolve la compattazione di memoria? Quali sono i difetti?

Problema risolto — frammentazione esterna: Nei sistemi con allocazione a partizioni dinamiche contigue, dopo molte allocazioni e deallocazioni la memoria si riduce a molti piccoli "buchi" sparsi, nessuno abbastanza grande per soddisfare una nuova richiesta di grandi dimensioni, anche se la memoria libera totale sarebbe sufficiente. La compattazione sposta tutte le partizioni occupate verso un estremo della memoria, unendo tutti i buchi in un unico grande blocco contiguo.

Difetti della compattazione:

1. **Costo computazionale elevato:** spostare il contenuto di tutti i processi richiede copiare grandi quantità di dati, proporzionale alla dimensione totale della memoria occupata — $O(N)$ dove N è la dimensione della RAM.
 2. **Sospensione obbligatoria dei processi:** i processi coinvolti devono essere sospesi per tutta la durata della compattazione (vedi es. 07/2025 per spiegazione dettagliata). Ciò causa pause nell'esecuzione del sistema.
 3. **Requisito hardware:** la compattazione è possibile solo se il sistema ha un meccanismo di rilocalizzazione hardware (es. registro di rilocalizzazione + limite, oppure MMU). Senza hardware di traduzione degli indirizzi, spostare un processo in memoria renderebbe invalidi tutti i puntatori interni al processo.
 4. **Impossibilità in alcune situazioni:** processi con trasferimenti DMA in corso (il DMA usa indirizzi fisici diretti) non possono essere spostati senza corrompere il trasferimento. Analogamente, processi con memoria condivisa hanno vincoli sulla rilocalizzazione.
 5. **Soluzione temporanea:** la frammentazione esterna si ripresenta dopo nuove allocazioni/deallocazioni. La compattazione deve essere ripetuta periodicamente.
-

Esame 28 maggio 2025

a) Revoca capability vs ACL (*vedi esame 15/01/2025 domanda d*)

La risposta è identica: la capability distribuisce il diritto tra i possessori del token, rendendo impossibile la revoca centralizzata. L'ACL centralizza il diritto nella risorsa, rendendo la revoca banale (rimuovere una entry).

b) Si può implementare la memoria virtuale con paginazione a richiesta senza TLB?

Sì, è **tecnicamente possibile**, ma le prestazioni sarebbero molto scarse.

Ruolo del TLB: il Translation Lookaside Buffer è una **cache hardware** per le entry della page table. Grazie al TLB, la traduzione da indirizzo logico a fisico avviene in pochi cicli (TLB hit rate tipicamente $> 99\%$). Senza TLB, ogni traduzione richiede accedere alla page table in memoria principale.

Paginazione a richiesta senza TLB — funzionamento:

1. La MMU consulta la page table in memoria per ogni accesso → **ogni accesso richiede almeno 2 accessi in memoria** (1 per la page table + 1 per il dato effettivo). Con page table a due livelli: 3 accessi.
2. Se la pagina è **non presente** (bit valid = 0), la MMU genera un **page fault** (trap al sistema operativo) esattamente come con il TLB. Il meccanismo di page fault non dipende dal TLB.
3. L'OS gestisce il fault: carica la pagina da disco, aggiorna la page table, riavvia l'istruzione.
4. La correttezza è **preservata**: il TLB è solo un'ottimizzazione, non è necessario per la

correttezza della paginazione.

Impatto sulle prestazioni:

- Senza TLB le prestazioni degradano di un fattore 2-3× su tutti gli accessi in memoria (non solo quelli che causano page fault).
- Con un hit rate TLB del 99% e costo miss 100 cicli, il costo medio per accesso senza TLB sarebbe enormemente superiore.

Conclusione: sì, la memoria virtuale con paginazione a richiesta si può implementare senza TLB. Il TLB esiste esclusivamente per rendere il meccanismo **praticamente utilizzabile**; la sua assenza non compromette la correttezza ma rende il sistema troppo lento per uso reale.

c) Perché il link fisico (hard link) non è implementabile in un file system FAT?

Hard link UNIX: una seconda directory entry che punta allo stesso i-node del file originale. L'i-node contiene il reference count; il file viene deallocato solo quando il contatore scende a zero.

Struttura FAT: non esiste un i-node. In FAT ogni directory entry contiene direttamente:

- Il nome del file
- Gli attributi (dimensione, data, permessi)
- Il numero del **primo blocco** della catena

Non esiste una struttura separata (equivalente all'i-node) condivisa tra più directory entry.

Perché l'hard link è impossibile in FAT:

1. **Nessun reference count:** FAT non ha un contatore di riferimenti associato ai dati. Se si creasse un secondo directory entry puntando allo stesso primo blocco, e poi si cancellasse il primo entry, FAT libererebbe l'intera catena di blocchi — non sapendo che un altro entry li usa ancora. Il secondo entry diventerebbe un **dangling reference** (punta a blocchi liberi o riassegnati).
2. **Nessuna struttura condivisa:** gli hard link funzionano perché due nomi puntano allo stesso i-node (metadati condivisi). In FAT i metadati (dimensione, attributi) sono nel directory entry: due entry non possono condividere uno stesso set di metadati.
3. **Doppia modifica dei metadati:** se si aggiornasse la dimensione del file (durante una scrittura) in un directory entry, l'altro entry rimarrebbe con il valore obsoleto.

Conclusione: l'assenza dell'i-node come struttura di metadati separata e condivisa rende strutturalmente impossibile implementare hard link in FAT.

d) Perché l'algoritmo del Banchiere è raramente usato nei sistemi reali?

L'algoritmo del Banchiere (Banker's algorithm) è teoricamente elegante per la deadlock avoidance, ma nella pratica ha numerose limitazioni:

1. **Conoscenza a priori del credito massimo:** richiede che ogni processo dichiari in anticipo il numero massimo di risorse che potrà richiedere durante la sua esecuzione. Nei sistemi operativi general purpose questo è impossibile: i processi non sanno a priori di quante risorse avranno bisogno.
 2. **Numero di processi e risorse fissati:** il Banchiere assume che il numero di processi e di classi di risorse sia costante. In un OS reale, i processi nascono e muoiono continuamente, e nuovi dispositivi possono essere aggiunti dinamicamente.
 3. **Complessità computazionale:** $O(n^2 \times m)$ per ogni richiesta di risorsa (n processi, m tipi di risorse). In un sistema con migliaia di processi e decine di tipi di risorsa, questo è eccessivo su ogni singola `malloc`, `open`, ecc.
 4. **Overhead su ogni allocazione:** ogni volta che un processo richiede anche una sola unità di risorsa, l'intero algoritmo di sicurezza deve essere eseguito, rallentando ogni operazione di allocazione.
 5. **Il deadlock è raro in pratica:** la maggior parte dei sistemi reali non entra in deadlock durante il normale funzionamento. Investire risorse computazionali per prevenire un evento raro non è giustificato.
 6. **Strategie alternative più semplici:** i sistemi reali preferiscono:
 - **Ignorare il problema** (Ostrich algorithm): se il deadlock è abbastanza raro, si fa ripartire il sistema.
 - **Rilevare e recuperare:** eseguire periodicamente il rilevamento (analisi del grafo di allocazione) e terminare i processi coinvolti solo quando il deadlock è effettivamente rilevato.
-

Esame 23 giugno 2025

a) In quali casi si preferisce C-LOOK a LOOK?

LOOK (Elevator): la testina si muove in una direzione servendo le richieste; quando non ci sono più richieste in quella direzione, inverte il moto.

C-LOOK (Circular LOOK): la testina si muove sempre nella stessa direzione; quando raggiunge l'ultima richiesta, salta direttamente alla prima richiesta nell'altra estremità del disco (senza servire nel ritorno).

LOOK è più efficiente in termini di distanza totale percorsa: la testina inverte direzione senza ritornare all'origine.

C-LOOK è preferito quando la priorità è l'equità:

1. **Distribuzione uniforme dei tempi di attesa:** in LOOK, i cilindri centrali vengono serviti più spesso (la testina li attraversa in entrambe le direzioni), mentre i cilindri agli estremi vengono serviti meno frequentemente. C-LOOK garantisce che ogni cilindro venga servito con frequenza proporzionale solo alla sua posizione nella coda, non alla sua posizione sul disco.

2. **Sistemi real-time o con requisiti di latenza massima garantita:** la distribuzione del tempo di attesa in C-LOOK è più uniforme e predicibile. Il worst-case waiting time in C-LOOK è $\leq$ tempo di un giro completo; in LOOK può variare di più.
 3. **Distribuzione non uniforme delle richieste:** se le richieste si concentrano su una metà del disco, LOOK passerebbe pochissimo tempo sull'altra metà. C-LOOK garantisce comunque un servizio periodico a tutte le aree.
 4. **Ambienti con fairness come requisito esplicito** (es. database con SLA): si preferisce la prevedibilità di C-LOOK alla sua minore efficienza in distanza.
-

b) Attacchi possibili senza il meccanismo del "sale" (salt) nelle password

Senza sale, le password sono memorizzate come `hash(password)`. Questo espone a:

1. **Attacco con dizionario (Dictionary Attack):** L'attaccante calcola in anticipo `hash(word)` per ogni parola di un dizionario di password comuni (milioni di voci). Confronta i valori precalcolati con gli hash nel file delle password. Tutte le corrispondenze sono trovate istantaneamente, senza tentare login sul sistema.
2. **Rainbow Tables:** Versione ottimizzata del dizionario: tabelle precalcolate che coprono *tutti* i possibili valori hash per password fino a una certa lunghezza/complessità, con tecniche di compressione (time-memory tradeoff). Con una rainbow table è possibile invertire quasi qualsiasi hash in pochi secondi.
3. **Rilevamento di password identiche:** Se due utenti hanno la stessa password, i loro hash sono identici. Un attaccante che vede gli hash sa immediatamente quali utenti condividono la password, e se la scopre per uno l'ha scoperta per tutti.
4. **Attacco offline su tutti gli account simultaneamente:** Una volta ottenuto il file degli hash, l'attaccante può lavorare offline, testando miliardi di combinazioni al secondo (GPU moderne), cercando contemporaneamente la password di tutti gli utenti.

Il salt risolve tutti questi problemi: `hash(password || salt)` dove salt è un valore casuale per ogni utente, memorizzato in chiaro accanto all'hash.

- Le rainbow tables diventano inutili (servirebbero tabelle separate per ogni salt possibile).
 - Password identiche producono hash diversi (salt diversi).
 - L'attaccante deve fare brute force separatamente per ogni account.
-

c) L'algoritmo del Banchiere evita anche la starvation?

No, l'algoritmo del Banchiere **non garantisce l'assenza di starvation**.

L'algoritmo garantisce solo la **safety** (il sistema non entra mai in deadlock), ma non la **fairness** (ogni processo ottiene le sue risorse entro un tempo finito).

Come può accadere la starvation:

1. Il processo P richiede risorse → l'algoritmo verifica se l'allocazione lascerebbe il sistema in stato safe → risposta: **unsafe** → P viene sospeso.
2. Nel frattempo, altri processi Q, R, S arrivano e fanno richieste che risultano safe → vengono serviti immediatamente.
3. Torna il momento di rivalutare P → di nuovo unsafe (altri processi hanno allocato risorse) → P continua ad aspettare.
4. Questo ciclo può ripetersi indefinitamente: P non entra mai in deadlock (il sistema è sempre safe senza P), ma P non viene mai servito.

Esempio concreto: un processo che richiede molte unità di una risorsa può attendere indefinitamente se arrivano continuamente nuovi processi che richiedono meno unità, mantenendo il sistema in uno stato dove la richiesta grande è sempre "unsafe".

Rimedi (non fanno parte dell'algoritmo base):

- Aging delle richieste sospese (aumentare la loro priorità nel tempo).
- Ordinamento FIFO delle richieste in attesa.
- Limite massimo al tempo di attesa prima di forzare la rischedulazione.

d) Perché usare spinlock in sistemi SMP? (vedi esame 15/01/2025 domanda b)

La risposta è identica: nei sistemi multiprocessore disabilitare gli interrupt è insufficiente (opera solo sulla CPU locale), mentre lo spinlock con `test&set` garantisce mutua esclusione atomica su tutti i processori con overhead minimo per sezioni critiche brevissime.

Esame 21 luglio 2025

a) Perché la compattazione richiede la sospensione dei processi coinvolti?

Durante la compattazione, la memoria fisica di un processo viene **copiata da un indirizzo fisico a un altro**. Se il processo fosse lasciato in esecuzione durante questo spostamento, si verificherebbero le seguenti situazioni critiche:

1. **Inconsistenza transitoria:** durante la copia, il vecchio indirizzo contiene ancora i dati (parzialmente copiati) e il nuovo indirizzo li contiene parzialmente. Se il processo legge in questa finestra temporale, potrebbe osservare dati parziali o corrotti.
2. **Program Counter e registri non aggiornati:** i registri della CPU (PC, SP, puntatori a dati) contengono ancora gli indirizzi fisici vecchi. Finché il registro di rilocalizzazione non viene aggiornato, il processo accede alla vecchia posizione — ma quella posizione contiene dati parzialmente sovrascritti dalla copia in corso.
3. **Race condition con la copia:** se il processo scrive in memoria mentre la copia è in corso, la

scrittura avviene alla vecchia posizione (già parzialmente copiata nella nuova). La copia non ha modo di sapere che quel dato è cambiato.

Sequenza corretta obbligatoria:

1. Sospendere il processo (salvarne il contesto).
2. Copiare l'intera memoria nella nuova posizione.
3. Aggiornare il registro di rilocalizzazione hardware (base register).
4. Riprendere l'esecuzione.

Solo quando i passi 2 e 3 sono completati atomicamente (o con il processo sospeso) è sicuro riprendere l'esecuzione.

b) Perché il nome del file non è nell'i-node?

In UNIX il nome del file è memorizzato nella **directory entry** (il mapping nome → numero_di_i-node), non nell'i-node stesso. Motivazioni:

1. **Hard link:** un singolo i-node può essere referenziato da più nomi in più directory (`ln fileA /dir1/x; ln fileA /dir2/y`). Se il nome fosse nell'i-node, ogni file potrebbe avere un solo nome. La struttura ad i-node permette N nomi → 1 i-node.
2. **Rinominare è un'operazione O(1):** `mv vecchio_nome nuovo_nome` modifica solo la directory entry (cambia la stringa del nome). Non serve accedere all'i-node né modificare i blocchi dati. Se il nome fosse nell'i-node, rinominare richiederebbe un accesso disco all'i-node.
3. **Separazione dei concetti:** l'i-node rappresenta il **file** (dati + metadati: permessi, owner, timestamps, lista blocchi). La directory rappresenta lo **spazio dei nomi** (mappatura nome → file). Sono concetti ortogonali: lo stesso file può avere nomi diversi in contesti diversi.
4. **Namespace multipli:** directory diverse possono contenere file con lo stesso nome che puntano a i-node diversi. Il nome è una proprietà del contenitore (directory), non del contenuto (file).

Operazione impossibile se il nome fosse nell'i-node: non sarebbe possibile implementare gli hard link, perché un i-node potrebbe avere un solo nome.

c) Microkernel: meno efficiente ma più flessibile e sicuro del kernel monolitico

Perché è meno efficiente:

In un kernel **monolitico** (es. Linux), tutti i servizi OS (file system, driver, rete) girano in kernel space. Una system call è:

```
user → [1 mode switch] → kernel (tutto in kernel space) → [1 mode switch] → user
```

In un **microkernel** (es. Minix, QNX), solo le funzioni minime (scheduling, IPC, gestione memoria

di base) sono in kernel space. I servizi (driver, file system) sono processi utente (server). Una system call equivalente richiede:

```
user → [1] → microkernel (IPC) → [2] → server (elaborazione) → [3] → microkernel (IPC) → [4] → user
```

2-4 mode switch invece di 2, più il costo dello scheduling e della copia dei messaggi IPC. Le prestazioni sono tipicamente 2-5× inferiori al monolitico per operazioni I/O intensive.

Perché è più flessibile:

- Un servizio (driver, file system) può essere avviato, fermato o **sostituito a caldo** senza riavviare il kernel.
- Aggiungere il supporto per un nuovo dispositivo significa scrivere un nuovo processo utente (server), non ricompilare il kernel.
- Versioni diverse dello stesso servizio possono coesistere.
- Portabilità: il microkernel è più piccolo e architettura-specifico solo per la parte minima; i servizi sono più facilmente portabili.

Perché è più sicuro:

- I servizi girano con **privilegi minimi** (principio di least privilege): un driver di rete non ha accesso alla memoria del file system.
- Un bug in un server (es. driver video) **non può corrompere il kernel** né la memoria di altri processi — al peggio, quel server crasha e può essere riavviato senza riavviare il sistema.
- **Superficie di attacco ridotta:** il kernel vero è minuscolo (poche migliaia di righe), più facile da verificare formalmente e da auditare.
- **Isolamento dei fault:** un guasto hardware gestito da un driver difettoso non abbatte il sistema operativo.

d) Per memorizzare le capability, serve crittografia a chiave singola o doppia?

Contesto: i processi utente memorizzano token di capability. Il sistema operativo (o un server di riferimento) deve poter verificare che una capability presentata da un processo sia autentica e non contraffatta.

Crittografia a chiave singola (simmetrica) — es. AES:

- La stessa chiave cifra e decifra.
- Il kernel crea `cifratura(dati_capability, chiave_segreta)` e lo dà al processo.
- Per verificare, il kernel decifra con la stessa chiave.
- **Funziona SE** solo il kernel possiede la chiave: il processo non può forgiare capability (non conosce la chiave).

- **Problema in sistemi con più server:** se più server (processi di sistema separati) devono verificare capability, tutti devono condividere la stessa chiave segreta → il segreto si distribuisce → rischio di compromissione.

Crittografia a doppia chiave (asimmetrica) — es. RSA:

- Il kernel firma le capability con la sua **chiave privata**: `firma(dati_capability, chiave_privata)`.
- Chiunque può verificare la firma con la **chiave pubblica**: `verifica(capability_firmata, chiave_pubblica)`.
- Il kernel non condivide mai la chiave privata: è impossibile forgiare capability senza di essa.
- Qualsiasi server può verificare autonomamente senza conoscere segreti.

Conclusione: occorre un **algoritmo a doppia chiave (chiave pubblica)**. La crittografia simmetrica potrebbe bastare se il solo kernel verifica le capability, ma non appena altri processi di sistema o server distribuiti devono verificarle, è indispensabile la crittografia asimmetrica: la chiave di verifica (pubblica) è distribuita liberamente, mentre la chiave di firma (privata) rimane esclusiva del kernel, garantendo che solo il kernel possa emettere capability valide.

Esame 5 settembre 2025

a) Quando è sconsigliato usare le ACL (Access Control List)?

Le ACL sono sconsigliate nelle seguenti situazioni:

1. **Numero elevato di soggetti autorizzati:** se una risorsa deve essere accessibile da migliaia di utenti/processi, la lista diventa enorme, costosa da mantenere ($O(n)$ per la ricerca) e difficile da gestire amministrativamente.
 2. **Sistemi distribuiti:** la ACL è memorizzata con la risorsa. In un sistema distribuito, ogni accesso richiede di consultare la ACL sul nodo che ospita la risorsa, introducendo latenza di rete. Le capability sono self-contained e possono essere verificate localmente.
 3. **Delegation dinamica:** se un utente vuole delegare temporaneamente un suo diritto a un altro soggetto, con ACL deve contattare l'amministratore per aggiungere un'entry. Le capability si delegano semplicemente passando il token (più flessibile).
 4. **Risorse con accesso anonimo o a soggetti dinamici:** quando il set di principal non è noto in anticipo (es. servizio web pubblico), mantenere ACL aggiornate è impraticabile.
 5. **Sistemi con requisiti di privacy dei principal:** una ACL rivela chi ha accesso a una risorsa; in alcuni contesti questo è indesiderabile.
-

b) Differenza fra funzione di libreria e system call

Funzione di libreria (es. `strlen`, `printf`, `malloc`):

- Eseguita in **user space**, nello stesso spazio di indirizzamento del processo chiamante.

- È una normale chiamata a funzione: push dei parametri sullo stack, salto all'indirizzo della funzione, return.
- **Nessun cambio di modalità** (nessun mode switch).
- Può accedere solo alle risorse accessibili al processo (memoria utente, file già aperti, ecc.).
- Implementata in librerie condivise (es. glibc).

System call (es. `read()`, `write()`, `fork()`, `open()`):

- Richiesta al **kernel** per eseguire un'operazione privilegiata.
- Richiede un **cambio di modalità**: user mode → kernel mode tramite una trap (istruzione speciale `syscall/int 0x80/svc`).
- Il kernel esegue l'operazione in modalità protetta con accesso diretto all'hardware.
- Al termine, il kernel ritorna in user mode con il valore di ritorno.
- Implementata nel kernel; la libreria C fornisce un wrapper thin (funzione C che prepara i parametri e invoca la trap).

Perché le system call sono necessarie:

1. **Protezione hardware**: operazioni come I/O diretto, modifica delle tabelle della memoria, gestione degli interrupt richiedono la modalità kernel (ring 0). Se i processi utente potessero eseguire queste istruzioni direttamente, un processo malevolo o buggy potrebbe corrompere l'intero sistema.
2. **Isolamento dei processi**: l'OS deve controllare e mediare tutti gli accessi alle risorse condivise (disco, rete, memoria). Senza system call, ogni processo potrebbe leggere/scrivere direttamente la memoria degli altri processi o il disco grezzo.
3. **Astrazione stabile**: la system call è un'interfaccia stabile che nasconde i dettagli dell'hardware specifico. Un programma scritto per Linux funziona su qualsiasi CPU supportata, perché non accede mai direttamente all'hardware.

Le funzioni di libreria spesso **wrappano** system call aggiungendo convenienza: `fwrite` aggiunge buffering e poi chiama `write`; `printf` formatta la stringa e poi chiama `write`.

c) Il deadlock può essere risolto con checkpointing & rollback?

Sì, il checkpointing & rollback è una valida tecnica di **recupero dal deadlock** (recovery), sebbene non ne prevenga il verificarsi.

Come funziona:

1. **Checkpoint**: periodicamente il sistema salva lo stato completo di un processo (contenuto della memoria, valori dei registri, file aperti e posizione corrente, risorse allocate) su memoria stabile (disco). La frequenza dei checkpoint determina quanto lavoro si perde in caso di rollback.
2. **Rilevamento del deadlock**: il sistema esegue periodicamente un algoritmo di rilevamento

(es. analisi del grafo di allocazione per la riducibilità completa). Quando rileva un deadlock, identifica l'insieme di processi coinvolti.

3. **Selezione della vittima:** si sceglie uno o più processi da fare rollback, minimizzando il costo (preferendo processi che hanno fatto poco lavoro dal checkpoint, o che tengono molte risorse).
4. **Rollback:** il processo selezionato viene riportato a un checkpoint precedente, rilasciando le risorse che teneva. Gli altri processi bloccati possono ora procedere.
5. **Ripresa:** il processo rolled-back ricomincia dal checkpoint, potendo potenzialmente fare meno richieste o in ordine diverso.

Considerazioni e limitazioni:

- Non tutte le operazioni sono reversibili: messaggi di rete inviati, output su stampante, scritture su DB senza transazioni non si possono "annullare" con il rollback.
- Un processo potrebbe essere selezionato ripetutamente come vittima → starvation (necessario tenere traccia della storia di rollback).
- Il costo di I/O per i checkpoint può essere elevato se i processi usano molta memoria.

Conclusione: sì, il checkpointing & rollback risolve il deadlock *dopo* che si è verificato (recovery), non lo previene. È una tecnica valida e usata in ambienti dove il costo del checkpoint è accettabile.

d) RAID0 vs RAID5: quale ha il tempo medio di accesso minore?

Con N dischi identici di bandwidth B e tempo di accesso casuale T (seek + rotational delay):

	RAID0	RAID5
Dischi con dati	N	N-1
Lettura sequenziale	$N \times B$	$(N-1) \times B$
Lettura random	$\approx T$ (1 disco per stripe) ma N parallel streams	$\approx T$ (1 disco per stripe) ma N-1 parallel streams
Scrittura sequenziale	$N \times B$	$\sim (N-1)/4 \times B$ (<i>write penalty</i>)
Scrittura random (1 blocco)	1 write su 1 disco	2 read + 2 write su 2 dischi

In lettura: RAID0 è leggermente più veloce (usa tutti N dischi, RAID5 usa N-1 per i dati). La differenza è piccola.

In scrittura: RAID0 è **significativamente più veloce**.

- RAID0: scrive il blocco su 1 disco → 1 I/O.
- RAID5: deve leggere il vecchio dato + leggere la vecchia parità + scrivere nuovo dato + scrivere nuova parità → **4 I/O su 2 dischi** (write penalty). La banda effettiva in scrittura random scende a circa 1/4 rispetto a RAID0.

Risposta: RAID0 ha il tempo medio di accesso minore (migliori prestazioni sia in lettura che in

scrittura). RAID5 paga il *write penalty* su ogni scrittura (4 operazioni invece di 1), rendendo le scritture molto più lente. In lettura la differenza è minore (un disco in meno per i dati in RAID5).

Però: RAID0 ha **tolleranza zero ai guasti** (il guasto di un solo disco distrugge tutti i dati). RAID5 tollera il guasto di 1 disco. Il vantaggio prestazionale di RAID0 si paga con l'affidabilità.

Risposte Esercizi G2 — Esami 2026

Sistemi Operativi — Renzo Davoli & Alberto Montresor, A.A. 2025/26 **Fonte:** presentazioni 00-intro.pdf – 08-sicurezza.pdf

Esame 09 gennaio 2026

a) Quali problemi possono accadere se si utilizza un file system ext2 dove un file ha un numero di riferimenti errato?

Ogni i-node in ext2 contiene un **contatore di hard link** (campo `i_links_count`) che indica quanti nomi nel file system puntano a quell'i-node. Quando scende a zero, il kernel libera i blocchi dati e l'i-node stesso.

Caso 1 — Contatore troppo alto (maggiore del reale): Anche dopo aver rimosso *tutti* gli hard link effettivi, il contatore non arriva a zero. Il kernel non dealloca mai l'i-node né i blocchi dati. Il risultato è una **perdita di spazio** permanente: il file è inaccessibile (non esiste nessun path che vi conduca) ma continua a occupare disco. `fsck` confronta il contatore nell'i-node con il numero reale di directory entry che vi puntano e corregge il valore.

Caso 2 — Contatore troppo basso (minore del reale): Questa è la situazione più pericolosa. Alla prima operazione di un link che porta il contatore (errato) a zero, il kernel **dealloca l'i-node e i suoi blocchi dati**, nonostante esistano ancora altri hard link validi. I processi o percorsi che continuano ad accedere al file tramite i link superstiti leggono blocchi che il file system può aver già riassegnato ad altri file, causando **corruzione di dati** su entrambi i file.

b) RAID1 vs RAID5: in quale array la lettura è statisticamente più veloce?

Consideriamo N dischi identici con banda singola B.

	RAID1 (mirroring)	RAID5 (striping + parità distribuita)
Dischi con dati utili	N/2 (gli altri N/2 sono mirror)	N-1 (1 "disco" equivalente per la parità, distribuita)
Banda sequenziale in lettura	$\approx (N/2) \times B$	$\approx (N-1) \times B$
Parallelismo random	Alta (ogni copia può servire la richiesta)	Alta (la parità non serve in lettura normale)

Risposta: RAID5 è statisticamente più veloce in lettura.

In RAID5 tutti gli N dischi contengono dati utili (la parità è distribuita, non dedicata a un singolo disco), quindi la banda aggregata in lettura sequenziale è proporzionale a $N-1$. In RAID1 con N dischi si hanno solo $N/2$ "stripe" indipendenti; anche se il controller può bilanciare le letture tra le copie, la capacità dati è la metà e la banda aggregata è al massimo $(N/2) \times B$. Per $N \geq 3$, $(N-1) > N/2$, quindi RAID5 vince.

c) La descrizione dell'algoritmo del banchiere è errata. Perché? Come si corregge?

La descrizione errata afferma:

"...se no l'operazione viene sospesa. **Quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare una o più operazioni sospese.**"

Errore 1 — il controllo alla restituzione: Quando un processo restituisce risorse, l'algoritmo del banchiere si limita ad aggiornare i vettori `Available`, `Allocation` e `Need`. **Non esiste nella formulazione standard una fase di ri-valutazione automatica** delle richieste sospese. I processi sospesi devono ri-emettere la richiesta esplicitamente; in implementazioni pratiche si risvegliano e ri-eseguono il test di sicurezza, ma la correzione non è automatica.

Errore 2 — "soddisfare una o più" senza test di sicurezza: L'espressione "il nuovo stato consente di soddisfare" suggerisce che sia sufficiente verificare se `Request ≤ Available`. Ma questo **non basta**: occorre eseguire l'intero **algoritmo di sicurezza** (trovare una sequenza sicura). Un sistema può avere risorse disponibili e trovarsi comunque in stato unsafe.

Errore 3 — "una o più operazioni sospese" in un colpo solo: Se si provano a soddisfare più richieste simultaneamente, il risultato potrebbe non essere safe anche se ciascuna singolarmente lo sarebbe. Le richieste vanno valutate **una alla volta**, rieseguendo il test di sicurezza dopo ogni concessione.

Descrizione corretta:

Quando un processo P richiede risorse R :

1. Se $R > \text{Need}[P]$ → errore.
2. Se $R > \text{Available}$ → P attende (il sistema non ha abbastanza risorse libere).
3. Tentare l'assegnazione: `Available -= R, Allocation[P] += R, Need[P] -= R`.
4. Eseguire l'algoritmo di sicurezza. Se lo stato è safe → l'assegnazione è confermata. Se non è safe → annullare la modifica e sospendere P .

Quando un processo rilascia risorse: aggiornare `Available`, `Allocation`, `Need`. I processi in attesa vengono risvegliati e **ri-emettono** la loro richiesta, che viene ri-valutata dal punto 1.

d) A cosa serve e come funziona il meccanismo di Aging negli scheduler a priorità dinamica?

Scopo: evitare la **starvation** — il rischio che un processo a bassa priorità non venga mai schedato perché continuano ad arrivare processi a priorità più alta.

Funzionamento: l'aging *aumenta progressivamente la priorità* di un processo in proporzione al tempo che trascorre in attesa nella ready queue senza ottenere la CPU. La regola tipica è:

$$\text{priorità_effettiva}(t) = \text{priorità_base} + f(\text{tempo_di_attesa})$$

dove f è una funzione crescente (lineare o con limite superiore). Dopo un tempo sufficiente, anche un processo con priorità base molto bassa raggiunge una priorità abbastanza alta da essere selezionato dallo scheduler.

Proprietà garantita: con aging ogni processo ottiene la CPU entro un tempo finito (upper bound dipende da priorità_base e velocità dell'aging), eliminando la starvation. Quando il processo ottiene la CPU, la sua priorità viene reimpostata al valore base.

Esame 06 febbraio 2026

a) In una tabella FAT possono esistere elementi di uguale valore (escluso il valore nullo EOF)?

Sì, possono esistere elementi di uguale valore, ma l'interpretazione e le conseguenze dipendono dal valore:

Casi leciti (valori uguali non problematici):

- **Marker EOC/EOF** (End Of Chain): più file terminano con lo stesso marker di fine catena. Tutti i blocchi finali di file diversi hanno lo stesso valore EOC nella FAT.
- **Marker BAD** (blocco difettoso): più blocchi difettosi condividono il medesimo valore speciale.

Caso illecito (valori uguali problematici): Se due **entry FAT di blocchi diversi** contengono lo stesso valore che rappresenta un puntatore a blocco dati ordinario (non EOC, non BAD), vuol dire che due catene distinte convergono verso lo stesso blocco — una situazione detta **cross-linked chains**. Il blocco condiviso risulterebbe essere contemporaneamente parte di due file diversi:

- Una scrittura su un file corrompe l'altro.
- La deallocazione di un file libera un blocco ancora in uso dall'altro.

fscck individua questa inconsistenza confrontando le catene e segnala il conflitto; la correzione tipica è duplicare il blocco condiviso assegnando una copia a ciascun file, ma almeno uno dei due risulterà corrotto.

b) Trovare un grafo di Holt con 5 nodi, risorse di molteplicità 2, tutti i processi in deadlock.

Configurazione: 3 processi (P1, P2, P3) + 2 risorse (R1, R2) = 5 nodi. Ogni risorsa ha 2 unità. Tutte le unità sono assegnate e nessun processo può procedere.

Assegnazioni e richieste:

Processo	Tiene	Richiede
P1	1 unità di R1	1 unità di R2
P2	1 unità di R2	1 unità di R1
P3	1 unità di R1 + 1 unità di R2	1 unità di R1 (seconda unità)

Verifica delle unità:

- R1: P1 tiene 1 + P3 tiene 1 = **2 unità su 2 allocate** → nessuna unità libera.
- R2: P2 tiene 1 + P3 tiene 1 = **2 unità su 2 allocate** → nessuna unità libera.

Verifica del deadlock:

- P1 attende R2: entrambe le unità di R2 sono tenute da P2 e P3, entrambi bloccati → **P1 bloccato**.
- P2 attende R1: entrambe le unità di R1 sono tenute da P1 e P3, entrambi bloccati → **P2 bloccato**.
- P3 attende la seconda unità di R1: entrambe le unità di R1 sono tenute da P1 e P3 stessa (già ha 1) → nessuno può cedere → **P3 bloccato**.

Il grafo di Holt **non è completamente riducibile**: nessun processo ha tutte le sue richieste soddisfatte, quindi nessuno può terminare e liberare risorse. Tutti e 3 i processi sono in deadlock. ✓

Rappresentazione del grafo (archi di assegnazione → e richiesta →):

```
R1[●●] → P1 (assegnazione: 1 unità a P1)
R1[●●] → P3 (assegnazione: 1 unità a P3)
R2[●●] → P2 (assegnazione: 1 unità a P2)
R2[●●] → P3 (assegnazione: 1 unità a P3)

P1 → R1 (richiesta di 1 unità di R2) [ERRORE: P1 richiede R2]
P1 → R2 (richiesta di 1 unità di R2)
P2 → R1 (richiesta di 1 unità di R1)
P3 → R1 (richiesta della seconda unità di R1)
```

(Corretto: P1 → R2, P2 → R1, P3 → R1)

c) Perché gli spinlock (test&set) sono fondamentali nei sistemi operativi multiprocessore?

Nei sistemi monoprocesore la mutua esclusione in kernel mode si realizza banalmente **disabilitando gli interrupt**: se un solo processore esegue, nessun altro thread può interferire finché gli interrupt sono disabilitati.

Nei sistemi **multiprocessore (SMP)**, questa tecnica è insufficiente per due ragioni:

1. **Disabilitare gli interrupt è locale:** ogni CPU ha il proprio registro di controllo degli interrupt. Disabilitarli sulla CPU A non impedisce alla CPU B di eseguire contemporaneamente codice kernel e accedere alle stesse strutture dati condivise. La mutua esclusione non è garantita.
2. **Le sezioni critiche del kernel sono brevissime:** usare semafori (che mettono il processo in sleep) richiede un context switch. Per sezioni critiche che durano decine di istruzioni, il costo del context switch è sproporzionato rispetto al lavoro da fare.

Lo spinlock con test&set risolve entrambi i problemi:

- L'istruzione test&set (o compare&swap) è **atomica a livello hardware** anche su bus multiprocessore: garantisce che due CPU non entrino nella stessa sezione critica simultaneamente.
- Il processo che trova il lock occupato esegue **busy-wait** (spin): cicla finché il lock si libera. Per sezioni critiche brevissime, il tempo di attesa è trascurabile e non si paga il costo del context switch.

Vincolo d'uso: gli spinlock sono appropriati *solo* per sezioni critiche molto brevi (poche istruzioni). Per attese lunghe si usano i mutex con sleep/wake, perché il busy-wait spreca cicli CPU.

d) Perché è difficile revocare un'autorizzazione fornita tramite capability?

Meccanismo capability: una capability è un *token* (struttura dati, numero, oggetto crittografato) che conferisce al possessore il diritto di accedere a una risorsa. Il diritto è incorporato nel token stesso, che può essere copiato e distribuito liberamente tra processi.

Problema di revoca — confronto con ACL:

	Access Control List (ACL)	Capability
Dove risiede il diritto	Centralizzato nell'oggetto (risorsa)	Distribuito presso ogni possessore
Revoca	Rimuovere l'entry dalla lista dell'oggetto → immediato, O(1)	Trovare e invalidare <i>tutte le copie</i> distribuite → difficile

Perché è difficile revocare una capability:

- La capability può essere stata **copiata**: un processo la riceve e la passa ad altri via `fork`, message passing, ecc. Non esiste un registro centrale di chi la possiede.
- **Non c'è punto unico di controllo:** per revocarla occorre propagare la revoca a tutti i possessori, ma il sistema non sa quanti e dove siano.
- Se la capability è un semplice numero o una struttura, il SO non può distinguere chi la usa legittimamente da chi la ha ricevuta illecitamente.

Soluzioni parziali (con i loro costi):

1. **Capability indirette:** la capability non punta direttamente alla risorsa ma a un oggetto intermedio (puntatore). Per revocare, si invalida l'oggetto intermedio. Aggiunge un livello di indirizzione a ogni accesso.
2. **Capability con scadenza (timestamp/nonce):** la capability contiene un numero di versione; alla revoca il SO incrementa il numero di versione della risorsa. Richiede un confronto a ogni accesso.
3. **Lista capability lato kernel:** il kernel mantiene tutte le capability emesse in una lista globale. Aggiunge overhead di gestione e un punto di contesa.

In tutti i casi la revoca è più costosa e complessa rispetto a una ACL, dove basta cancellare una riga da una lista centralizzata.